PROJECT DEMO GUIDE

Messify

Fresh mess food, one tap away. A web platform that connects students and working professionals in Nagpur with home-style mess and tiffin kitchens.

What it is • Tech stack • First-time setup • Step-by-step demo flow for every user type

Messify • Shradha Park, B-37-39/1, Wadi Link Road, MIDC, Hingna, Nagpur 441110 • (+91) 82386 92705

1. What is Messify?

Students in hostels and working professionals usually find a mess through word of mouth, pay in cash, and have no idea whether lunch will actually arrive. **Messify puts that whole routine online**: find a mess, pick a plan, pay, skip a meal when you travel, and watch the tiffin come to you on a live map.

The problem and the solution in one line each

Problem today	How Messify solves it
Hard to find a trusted mess nearby	Browse verified messes with today's real menu and price
Cash, paper booklets, forgotten dues	Pay by UPI QR or Cash on Delivery; automatic monthly bill
Paying for meals you did not eat	One-tap **Skip Meal**; the amount is credited back in the bill
"Where is my tiffin?"	Live delivery tracking on a map
Mess owners track everything on paper	Menu, payments, subscribers and delivery in one dashboard

Four kinds of users (four "panels")

Student

Browse messes, order a plan, pay, skip meals, ask for custom food, track delivery, see the bill.

Mess Owner

Posts the daily menu and prices, approves UPI payments, sees subscribers, links delivery partners, creates routes.

Delivery Partner

Gets the day's route, starts it, shares live location, marks each stop delivered, scans tiffin box QR codes.

Admin

Sees everything on the platform in real time: users, orders, payments, deliveries, with search and CSV export.

Main features

Live mess menus · **Weekly / Monthly / Lunch-only plans** · **UPI QR payment** · **Cash on Delivery** · **Coupons** · **Skip meal + auto credit** · **Custom food request** · **Live delivery map** · **Tiffin box QR** · **Browser location detection** · **Notifications** · **Messify Assistant (help bot)** · **Admin realtime reports**

2. What we used (tech stack, in simple words)

Think of the app as a **website that runs in the browser** and talks to **Google Firebase** for logins and data. There is no separate server to maintain.

Tool	Where it is used	Plain-English meaning
React 18	All screens	A library for building the pages as reusable pieces (button, card, sidebar). Pages update instantly without reloading.
Vite	Dev server and build	The tool that runs the project while developing (`npm run dev`) and packs it for the web (`npm run build`).
Tailwind CSS	Styling	A way to style with small ready-made classes; holds the Messify colors (forest green, saffron, gold).
React Router	Page navigation	Gives each page its own address (`/login` , `/student` , `/mess`) and protects pages by role.
Firebase Authentication	Login / signup	Google's service that stores email + password safely. We never store passwords ourselves.
Cloud Firestore	The database	Stores all data (users, menus, orders, payments...). It is **realtime**: when one person changes data, every open screen updates by itself.
Firestore Security Rules	`firestore.rules`	The "door guard": a student can only read their own orders, a mess owner only their own payments, etc.
Leaflet + OpenStreetMap	Live delivery map	Free map library and free map tiles (no Google Maps bill).
Geolocation + Nominatim	"Detect my location"	The browser gives GPS coordinates; Nominatim (OpenStreetMap) turns them into a street address.
qrcode.react / html5-qrcode	Tiffin box QR	Draws the QR code for each tiffin box and scans it with the phone camera.
date-fns	Dates	Small helper for "valid till 31 Oct", "22 days remaining".
firebase-admin (scripts)	`npm run seed` / `clean`	A developer-only script that creates demo accounts and sample data, or wipes test data.
Firebase Hosting	Going live	Where the built website is published (`firebase.json` is already set up).
Cloudinary (optional)	Images	If not configured, payment screenshots are compressed and kept inside Firestore.

2b. How the pieces fit together

How the pieces fit



> **Realtime in one sentence:** the mess owner's screen "listens" to Firestore, so when a student orders or skips a meal, the mess owner's numbers change on their screen within a second, with no refresh button.

Firestore tables (called "collections")

`users` • `messProfiles` • `menus` • `subscriptions` (orders) • `payments` • `skips` • `ledger` (monthly bill) • `customRequests` • `routes` (delivery) • `tiffinBoxes` • `notifications` • `messDeliveryPartners`

Where things live in the project

```
src/pages/          Student, Mess, Delivery, Admin screens + Landing, Login, Signup
src/components/     Shared pieces (layouts, map, assistant, buttons)
src/firebase/       Firestore and Auth functions (the "data layer")
src/utils/          Pricing, location, assistant knowledge
firestore.rules     Who can read/write what
scripts/seed.mjs    Demo data     scripts/clean.mjs   Wipe test data
FIREBASE_SETUP.md   Step-by-step Firebase setup
```

3. First-time setup (do this once)

You need Node.js (v18 or newer) installed, and access to the Firebase project **messify-prod**. The Firebase keys are already inside `src/firebase/config.js`.

A. Turn on Firebase (one time, in the browser)

1. **Authentication** → Get started → enable **Email/Password**.
2. **Firestore Database** → Create database → production mode → location `asia-south1` (Mumbai).
3. Firestore → **Rules** tab → paste the contents of `firestore.rules` → **Publish**.
4. Project settings → Service accounts → **Generate new private key**. Save it as `scripts/serviceAccountKey.json` (keep it private; never put it in `public`).

B. Run the project on your computer

```
cd AI_Mess_Management_SourceCode
npm install          # downloads the libraries (first time only)
npm run seed         # creates demo accounts + sample data
npm run dev          # starts the website at http://localhost:5173
```

C. Handy commands

Command	What it does
`npm run dev`	Run locally for development / demo
`npm run build`	Create the production website in the `dist` folder
`npm run seed`	Create / refresh demo accounts and sample data (safe to repeat)
`npm run clean`	Dry run: shows what test data would be deleted
`npm run clean -- --yes`	Really delete orders, payments, skips, bills, routes, boxes, notifications. Keeps logins, messes, menus.

D. Demo logins (created by `npm run seed`)

Role	Email	Password	Sign in at
Student	student@messify.in	Student@123	/login → Student tab
Mess owner	mess@messify.in	Mess@123	/login → Mess Owner tab
Delivery	delivery@messify.in	Delivery@123	/login → Delivery tab
Admin	admin@messify.in	Admin@123	/admin/login

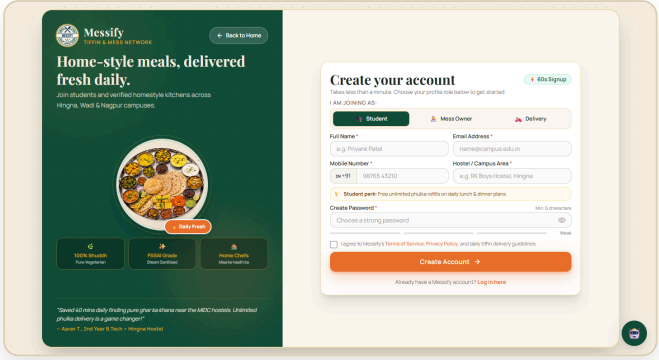
> **Tip:** in `npm run dev` the login page shows one-click buttons that fill these logins. They do not appear in the production build. **Delete the demo accounts before real users sign up**, because the passwords are public.

4. Demo flow: a brand-new student, start to finish

This is the story to tell colleagues: "*Aarav just moved into a hostel in Hingna and wants lunch and dinner.*"

1 Landing	2 Sign up	3 Choose	4 Checkout	5 Use
See messes	Create account	Mess + plan	UPI or COD	Skip, track

- **Landing page** (`/`). Shows the detected location at the top, today's menu from every real mess, plans, coupons and FAQs. All numbers here are live from the database.
- **Sign up** (`/signup`). Pick **Student**, fill name, email, mobile, hostel area, password, tick the terms. They land on their dashboard.
- **Browse Messes.** Search by name or area; each card shows rating, area and today's lunch with price.
- **Mess page → pick a plan:** Weekly (7 days), Monthly (30 days) or Monthly Lunch-only. Price comes from that mess's own menu with a bundled ~8% discount.
- **Checkout.** Add address (or tap "Use my current location"), landmark, lunch/dinner slot, dietary note. Optionally add coupon `MESSIFY50` . An account is required first.



Signup page with role tabs (Student / Mess Owner / Delivery)

Two ways to pay (the key demo moment)

Cash on Delivery

Order is **confirmed immediately**. The student pays the delivery partner when the tiffin arrives.

Pay Online (UPI QR)

Scan the mess owner's QR, pay in GPay / PhonePe / Paytm, then submit name + UTR number + screenshot. Plan turns **Active** after the mess owner approves.

After the order (student dashboard)

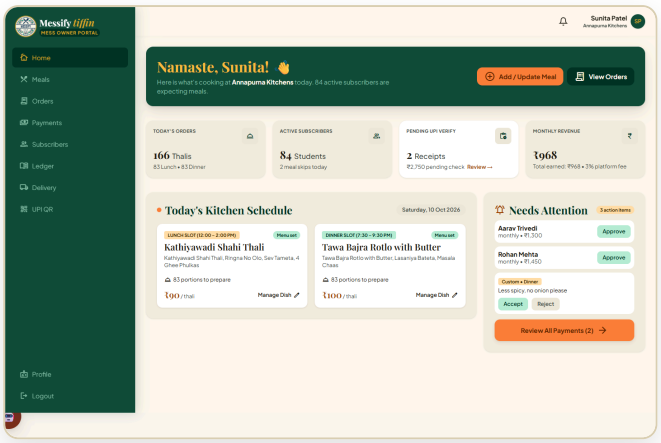
- **Skip Meal** for today (lunch or dinner): the amount is credited in the monthly bill, and the mess sees it.
- **Custom Request**: "less spicy, no onion". The mess owner accepts or rejects; status shows on the card.
- **Live Tiffin Tracking**: Preparing → On the way → Arriving → Delivered, with a map once the rider shares location.
- **Billing** page: every meal, skip and payment this month. **Bell icon**: notifications.

5. Demo flow: mess owner and delivery partner

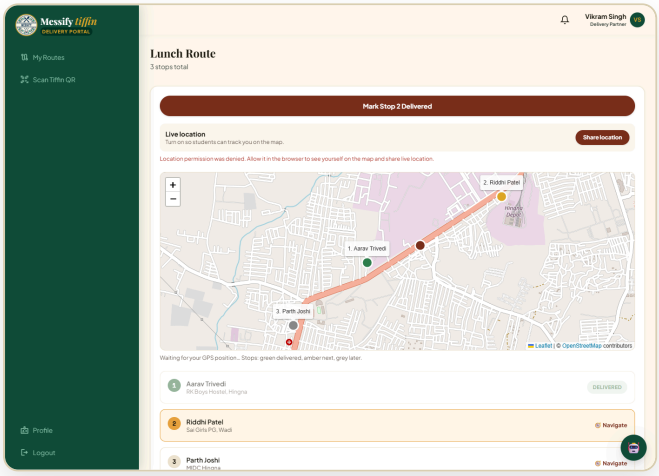
MESS OWNER

Sign in at `/login` → Mess Owner tab.

1. **Home** shows live numbers: today's orders, active subscribers, pending UPI payments, monthly revenue.
2. **Meals:** type today's lunch and dinner items and price → Save. Students see it at once.
3. **Payments:** check the UTR and screenshot → **Approve** (plan activates) or Reject with a reason.
4. **UPI QR:** upload the QR students pay to.
5. **Delivery:** link a delivery partner by their email, then create a lunch/dinner route with the students' stops.
6. **Ledger:** confirmed payments, the 3% platform fee, and what the mess receives.



Mess owner home (sample numbers)



Delivery partner route page with live map (sample stops)

DELIVERY

Sign in at `/login` → Delivery tab.

1. **My Routes:** open today's lunch or dinner route.
2. **Start Route** and allow location: the map shows your position and numbered stops.
3. **Share location** so students can follow you live.
4. **Mark Stop Delivered** at each door; **Call** or **Navigate** buttons help find the house.
5. **Scan Tiffin QR**: scan the box on delivery and again on return.

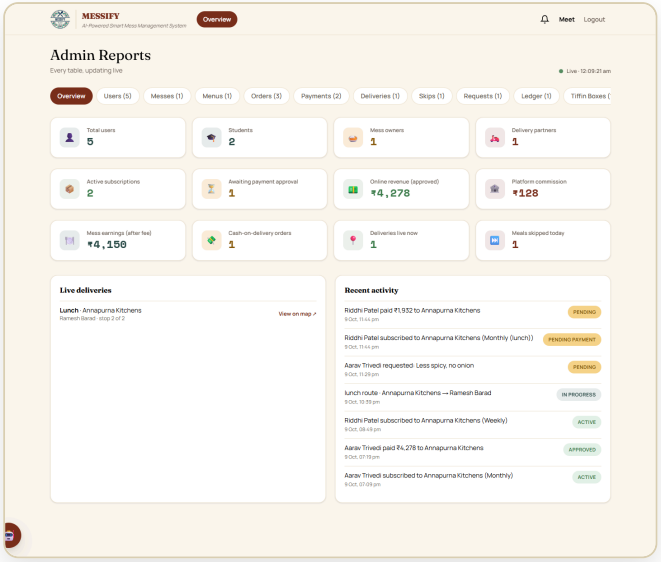
> Phones give GPS only on **HTTPS** (or localhost). Keep the route page open while delivering.

6. Admin console and the help assistant

ADMIN /admin/login

- Overview cards: users, active subscriptions, online revenue, platform commission, deliveries live now.
- **12 tabs**, one per database table, all updating in real time.
- Search every table and **Export CSV**.
- "Live deliveries" lists routes in progress with a map link.

> **Great demo trick:** open the admin console on one screen and place an order as a student on another. The new order appears on the admin screen within a second.



Admin reports (sample data)



Landing page with the Messify Assistant open

Messify Assistant (robot button)

A small help chat at the bottom-right of every page. It is **not a general AI**: it answers from Messify's own rules (plans, payments, coupons, skipping, tracking, logging in) and gives different help to students, mess owners and delivery partners.

When signed in it can also look up live data, for example "*What is my subscription status?*" or "*How many subscribers do I have?*".

Try asking: "Is cash on delivery available?", "How do I skip a meal?", "Today's menu".

Login, signup and navigation

- Login and signup have a **Back to Home** button (or **My Dashboard** when already signed in).
- Wrong portal? The login page tells you which tab your account belongs to.
- Forgot password: type the email, tap "Forgot password?"; a reset link is emailed.

7. A 10-minute live demo script

Use **two browser windows**: Window A = student (normal), Window B = mess owner (incognito, so both stay signed in). Optional third window = admin.

Time	Window	What to do and say
0:00	A	Open the landing page. "Everything here is live: these menus come straight from the mess owners." Point at the location pill and the assistant button.
1:30	A	Click Login → Student tab → **Student** demo button → Log in. (Or sign up as a new student to show the form.)
2:30	A	Browse Messes → open a mess → choose **Monthly** → Checkout. Apply `MESSIFY50`. Choose **Pay Online**, upload any image as proof, submit.
4:00	B	Mess owner login. The Needs Attention card already shows the new payment. Click **Approve**. "The student gets a notification and the plan activates."
5:00	B	Meals → change today's lunch price → Save.
5:30	A	Back on the student dashboard: the plan is Active and the new price is visible without refreshing. Tap **Skip Dinner**. Send a custom request "less spicy".
6:30	B	Home numbers changed (skip counted). Accept the custom request.
7:30	C	Delivery login → open the route → **Start Route** → Share location → Mark stop delivered.
8:30	A	Live Tiffin Tracking moves to "On the way" / "Delivered" on its own.
9:00	D	Admin console: every table updated; show search and Export CSV.

Questions colleagues may ask

Is the money handled by the app?	No. Online payment is a **manual UPI** flow: the student pays the mess's own QR and the mess owner verifies it. There is no payment gateway yet.
How does Messify earn?	3% commission on each approved online payment (the mess receives 97%). Cash on Delivery is not counted.
Is student data safe?	Logins use Firebase Authentication. Security rules limit each role to its own data. Routes show students' addresses only to the mess, the assigned rider and the students on that route.
Does it work on phones?	Yes: desktop, tablet and phone layouts. Live GPS needs HTTPS.
Is the assistant real AI?	No, it is a rule-based help bot trained on this project's rules. A real AI model would need a small backend to protect the API key.

8. Current limits and small glossary

Honest limits (good to say before someone asks)

- UPI payments are **verified manually** by the mess owner; no automatic gateway or refunds.
- Live delivery location works only while the rider keeps the route page open.
- The "login security code" (2FA) box on the admin login is visual only.
- Coupons are not yet limited to one use per student.
- Landing page counts messes and menus only; subscriber numbers are private so are not shown publicly.
- Demo accounts and their passwords are public in this guide: remove them before going live.
- Some inner pages (Subscriptions, Billing, Profile, Payment) still carry the older card styling inside the new layout.

Words you may hear

Frontend	The part users see and click (the React website).
Backend / BaaS	The server side. Here Firebase provides it, so we wrote no server code.
Collection / document	A Firestore "table" / one "row" in it.
Realtime listener	The screen subscribes to data and is updated automatically when it changes.
Security rules	Rules in `firestore.rules` that decide who may read or write each collection.
Seed	Fill the database with sample data for testing.
Subscription	An order for a meal plan (weekly / monthly) at one mess.
Ledger	The monthly bill: meals charged, skips credited, payments made.
UTR	The transaction reference number shown in a UPI app after paying.
Route / stop	A delivery run for one meal / one student's drop-off point on it.
Build / deploy	Pack the app for production / publish it to Firebase Hosting.